

BOS-2L: Tolerance-Preserving Right Shift with Acceptance-Gated Recovery

Chen-Hong

Department of Computer Science and Information Engineering
National Central University, Taoyuan, Taiwan

Abstract—A material delay need not trigger makespan optimization. We revise bottleneck-oriented overlap scheduling into a tolerance-first architecture: freeze executed history, project the remaining plan forward, and recover only if the projection violates a delivery budget or an operational constraint. In an integer checkpoint model, we prove fixed-order projection feasibility and minimality, distinguish triggers from infeasibility certificates, and implement comparable greedy, dynamic-programming, and LP-assisted recovery. DP preserves segment nonpreemption; LP expected starts determine executable-set priorities. Across 144 synthetic events and three delivery budgets, projection accepts 156 of 432 scenarios without recovery. Greedy recovery raises the on-time count to 200; tested DP and LP portfolios do not increase that count. A separate 80-instance suite checks full DP against MILP. Inputs, schedules, statuses, code, and verification records accompany the manuscript. These results support selective recovery, not universal superiority of DP or LP.

Index Terms—overlap scheduling, schedule repair, tolerance, dynamic programming, linear programming

I. WHY PRESERVE AN ACCEPTABLE PLAN?

Production commitments concern acceptable delivery and executable operations, not only the smallest achievable makespan. A delay can consume reserved slack while the plan remains acceptable. Reassigning machines or repeatedly optimizing such a plan can create avoidable operational change. Our policy is therefore

$$\begin{aligned} \text{delay} &\rightarrow \text{forward projection} \rightarrow \text{acceptance test} \\ &\rightarrow \text{recovery if required.} \end{aligned} \quad (1)$$

The acceptance test precedes any recovery optimization. The method deliberately permits a longer makespan than an always-reoptimize method when both meet the delivery budget.

Predictive-reactive scheduling and right-shift repair are established ideas, including the strategies and policies framework of Vieira et al. [1] and reactive job-shop repair [2]. Our contribution is a specific integration with BOS progressive handoffs and frozen-history checkpoint semantics, together with checkable projection and trigger statements and a reproducible comparative implementation. We do not claim invention of right shifting, slack, dynamic programming, or temporal precedence. General temporal resource constraints and list scheduling provide relevant foundations [3], [4], [5].

A. Version Lineage and Corrections

The September 15 BOS model supplies nominal dispatch; September 28 LP supplies checkpoint transformation. The

earlier progress DP allowed arbitrary slot preemption. We now require uninterrupted processing within each segment, aligning DP with LP permissions; its earlier optimum is not transferred to this model.

We retain five corrections: all incoming handoff conditions have AND semantics; wall-clock passage is not production progress during pauses; history is fixed for all comparisons; heuristic outputs need not be monotone in disruption; and a fractional LP solution is not a feasible schedule. The nominal list-scheduling approximation is not automatically inherited by intentionally idling right-shift schedules.

II. MODEL: NOMINAL PLAN, HISTORY, AND CHECKPOINTS

A. Nominal BOS

Let $G = (V, E)$ be a DAG with processing requirements $p_i > 0$, releases $r_i \geq 0$, and m identical unit-speed machines. All data are integer in the implementation. For $(i, j) \in E$ define

$$\ell_{ij} = p_i - \delta_{ij}, \quad 0 \leq \delta_{ij} \leq p_i. \quad (2)$$

A nominal nonpreemptive schedule satisfies $s_i \geq r_i$, $s_j \geq s_i + \ell_{ij}$ and disjoint intervals $[s_i, s_i + p_i)$ on each machine. BOS chooses the largest score among executable jobs, where

$$b_i = \max\{p_i, \max_{(i,j) \in E} (\ell_{ij} + b_j)\}. \quad (3)$$

Scores are computed in reverse topological order. All incoming conditions must hold. “Partial-kit advance” means that downstream production may start after every required predecessor reaches its specified progress threshold; it does *not* mean that an arbitrary subset of missing mandatory components may be ignored. A true subset/OR bill-of-materials model needs additional activation variables and is outside this paper.

Let $S^0 = (s^0, \mu^0)$ denote the nominal plan and $C_0 = \max_i (s_i^0 + p_i)$. At event time $t_0 < C_0$, the nominal execution prefix is frozen. Its progress is

$$q_i^0 = \min\{p_i, \max\{0, t_0 - s_i^0\}\}. \quad (4)$$

The real implementation must supply observed progress when execution differs from this assumed prefix. An event d gives material gates (i, g_i, a_i^d) : work beyond progress g_i requires arrival at absolute time a_i^d , with $q_i^0 \leq g_i < p_i$ and $a_i^d \geq t_0$. A gate already crossed before revelation cannot retroactively invalidate history.

B. Residual Representation

Cut every unfinished job at q_i^0 , each outstanding outgoing handoff threshold, its material gate, and p_i . Each positive interval $[a, b]$ becomes a residual segment $v = (i, a, b)$ with duration $h_v = b - a$. Consecutive segments of one job have finish-start arcs. For an unstarted successor job, each unsatisfied positive handoff points from the segment that ends at that threshold to the successor's first segment. Zero-lag handoffs require predecessor start no later than successor start and retain a zero-lag arc. Satisfied gates are removed. Segments beginning at a material gate have release at least $a_i^d - t_0$; original releases are also retained where applicable.

Write the residual graph $G_R = (V_R, E_R)$, $N = |V_R|$, segment releases R_v^d , and arc lags $\lambda_{uv} \in [0, h_u]$. All residual times x_v are relative to t_0 . Segment constraints are

$$x_v \geq R_v^d, \quad x_v \geq x_u + \lambda_{uv}. \quad (5)$$

On any one machine, all intervals $[x_v, x_v + h_v]$ must be pairwise disjoint. A segment cannot be interrupted. Migration and suspension are allowed at the declared segment boundaries, including the event boundary, with zero costs. One original job never runs on two machines simultaneously because its segment chain is sequential. These permissions apply to every recovery method. The reference right shift voluntarily preserves nominal machines and orders. Machine failures, switching costs, maximum waiting lags, and no-wait operations are not in the primary experiment.

Lemma 1 (Progress-consistent reconstruction). *Every feasible residual schedule joined to the frozen prefix respects original work amounts, all progress handoffs, material gates, job non-parallelism, and machine capacity under the stated checkpoint permissions.*

Proof. The segments partition exactly the unprocessed interval $[q_i^0, p_i]$ of each job. Their chain prevents overlap and reversal. An unsatisfied positive handoff can release a successor only after the segment ending at that work threshold completes; the preceding chain guarantees that all earlier work was performed. A zero-lag arc enforces start order, including simultaneous starts. Previously satisfied handoffs stay satisfied because completed work is retained. Segment releases enforce material gates, and residual intervals begin at or after t_0 , disjoint from the historical prefix. Residual machine nonoverlap completes the proof. $\square$

The validator reconstructs job processing intervals independently of the transformed arcs and measures actual predecessor work completed by a successor's first start.

III. FORWARD STATE PROJECTION AND ITS BOUNDS

The undisturbed reference start for residual segment $v = (i, a, b)$ is

$$\bar{x}_v = \max\{t_0, s_i^0 + a\} - t_0, \quad \bar{\mu}_v = \mu_i^0. \quad (6)$$

Augment G_R by a machine arc from each segment to the next segment on its reference machine, with weight equal to the

earlier segment's duration. Denote this labeled arc graph by A_0 . Parallel precedence and machine constraints use the larger weight for computation, while their distinct meanings remain valid. The reference is feasible for these arcs. Positive-weight cycles would contradict its feasibility; any all-zero cycle would have to use only original DAG arcs since every machine arc has positive weight. Thus A_0 is acyclic.

Define the forward projection in its topological order:

$$x_v^d = \max\{\bar{x}_v, R_v^d, \max_{(u,v) \in A_0} (x_u^d + w_{uv})\}. \quad (7)$$

This is propagation of lower bounds under fixed decisions, not optimization over machine orders. Let

$$C_{\text{shift}} = t_0 + \max_v (x_v^d + h_v), \quad (8)$$

$$\Delta_d = \max_v (R_v^d - \bar{x}_v)_+, \quad U_d^{\text{shift}} = C_0 + \Delta_d. \quad (9)$$

Uniform right shift uses $\bar{x}_v + \Delta_d$ on the original residual machines. Projection and uniform shift are distinct policies: projection can absorb a delay in internal gaps instead of moving every residual segment by the same amount.

Theorem 2 (Feasibility and minimality of right shift). *With finite delayed releases, fixed history, the declared checkpoint permissions, and only the lower-bound constraints in (5), projection is feasible and is the componentwise least schedule preserving reference machine orders and the floors $x_v \geq \bar{x}_v$. Moreover,*

$$C_0 \leq C_{\text{shift}} \leq C_0 + \Delta_d. \quad (10)$$

Proof. Topological evaluation enforces all releases, reference floors, lag arcs, and successive machine nonoverlap arcs. Nonoverlap of every consecutive pair implies nonoverlap of all intervals on that machine. Lemma 1 establishes physical feasibility. For any other feasible fixed-order schedule y satisfying the floors, induction gives $y_v \geq x_v^d$: it must exceed the same release and floor and each $y_u + w_{uv} \geq x_u^d + w_{uv}$. Uniformly translating the reference by Δ_d satisfies all releases by its definition and leaves every start difference and machine separation unchanged. It is therefore such a feasible y . Finally at least one unfinished job finishes at C_0 in the reference, so the reference floors imply the lower inequality. $\square$

The theorem does not allow shifting executed work. If preemption of a running job at t_0 is forbidden, one must hold that active interval fixed and model its remaining reservation. The present all-residual shift is not a theorem for that different model. Likewise, additional upper-bound constraints may fail after a shift and must be explicitly checked.

A. State Transition and Monotonicity

A decision state is $X_d = (t_0, H_{t_0}, G_R, R^d, \bar{x}, \bar{\mu}, B)$. With fixed segments and reference decisions,

$$X_d \xrightarrow{R^{d'} \geq R^d} X_{d'}, \quad x^{d'} = \Pi_{A_0, \bar{x}}(R^{d'}). \quad (11)$$

Induction in (7) gives $x^{d'} \geq x^d$. At the same fixed history and with identical permitted operations, increasing gate arrivals

shrinks the physical feasible set $\mathcal{F}_{d'} \subseteq \mathcal{F}_d$. Consequently $\text{OPT}_{d'} \geq \text{OPT}_d$. This concerns environments at one decision epoch. It does not compare remaining makespans measured from different epochs, nor arbitrary outputs of heuristics that change decisions. If gate positions change, compare physical feasible sets with a common refinement of checkpoint permissions before invoking monotonicity.

B. One Benchmark and Two Different Upper Bounds

Define OPT_d as the minimum *absolute* completion time among schedules sharing H_{t_0} and the declared checkpoints, but free to change future orders and machines. For a residual path $P = (v_1, \dots, v_k)$ define

$$W_R = \sum_v h_v, \quad (12)$$

$$L_{R,\lambda} = \max_P \left(R_{v_1}^d + \sum_{j < k} \lambda_{v_j v_{j+1}} + h_{v_k} \right), \quad (13)$$

$$\text{LB}_d^{\text{ana}} = t_0 + \max\{W_R/m, L_{R,\lambda}\}. \quad (14)$$

Workload and path propagation independently lower-bound every completion time; their maximum, not their sum, is a valid bound. A solved LP may strengthen this to $\text{LB}_d = t_0 + \max\{W_R/m, L_{R,\lambda}, z_{LP}\}$.

$$\boxed{\text{LB}_d \leq \text{OPT}_d \leq C_{\text{shift}} \leq U_d^{\text{shift}}}. \quad (15)$$

This is the requested chain under Theorem 2. After recovery, a feasible incumbent gives another bound $U_d^{\text{best}} \leq C_{\text{shift}}$ if shift remains an admissible candidate. It must *not* replace U_d^{shift} on the right side of (15): the best candidate can finish earlier than shift. If an added operational constraint invalidates shift, the middle inequality using C_{shift} no longer follows for that augmented feasible set. Only validated candidates supply its upper bounds.

IV. TOLERANCE AND RECOVERY TRIGGER

Let D be the nominal committed delivery time and $\tau \geq 0$ a delivery tolerance, both in time units:

$$B = D + \tau, \quad C_0 \leq D, \quad \sigma_0 = B - C_0. \quad (16)$$

The revelation time is t_0 , not τ . In experiments $D = C_0$. Consumed and remaining schedule slack are

$$a_d = C_{\text{shift}} - C_0, \quad \sigma_d = B - C_{\text{shift}} = \sigma_0 - a_d. \quad (17)$$

This is delivery slack, distinct from overlap allowance δ_{ij} , arrival lateness, and any dimensionless approximation parameter.

Let $\text{Valid}_d(S)$ check physical constraints, fixed history, and any operational predicates supplied to the policy. Then

$$\text{Accept}_d = \text{Valid}_d(S_{\text{shift}}) \wedge (C_{\text{shift}} \leq B), \quad (18)$$

$$\boxed{\text{Trigger}_d} = \neg \text{Valid}_d(S_{\text{shift}}) \vee (C_{\text{shift}} > B). \quad (19)$$

Theorem 3 (Acceptance and trigger semantics). *If Accept_d holds, returning the projected schedule meets every tested acceptance condition without recovery. Otherwise recovery is*

required by this policy. A trigger alone does not prove no acceptable schedule exists. If a valid lower bound satisfies $\text{LB}_d > B$, no schedule in the benchmark feasible set can meet B .

Proof. The first two statements follow from the acceptance conjunction and its logical negation. The trigger tests a particular fixed-order plan, whereas recovery may change decisions. For the last statement, every feasible completion satisfies $C \geq \text{OPT}_d \geq \text{LB}_d > B$. $\square$

The negation $\text{LB}_d \leq B$ is not a feasibility certificate. Conversely, a verified candidate with $C \leq B$ is a feasibility certificate, even if no optimum has been found. No selected recovery method is guaranteed to rescue every triggered event.

For a fixed gate pattern and a scalar delay magnitude d , define

$$d_{\text{shift}}^*(B) = \sup\{d : C_{\text{shift}}(d) \leq B\}, \quad (20)$$

$$d_*^*(B) = \sup\{d : \text{OPT}_d \leq B\}. \quad (21)$$

When releases are nondecreasing in d , and the operational model is unchanged, projection and optimum are nondecreasing. Feasible delay sets are downward closed and $d_{\text{shift}}^*(B) \leq d_*^*(B)$. Uniform-shift condition $\Delta_d \leq \sigma_0$ is sufficient for acceptance in the primary model; projection can accept more because it uses local gaps. In a deadline-fixed delay-maximization LP, relaxation gives an *upper* bound on tolerable delay, not a realizable tolerance until a schedule is checked.

V. COMPARABLE RECOVERY METHODS

A. Greedy Repair and Safeguards

The residual BOS dispatcher schedules executable segments by overlap-adjusted score, respecting zero-lag same-time closure. Greedy recovery retains the better feasible candidate among projected and greedy schedules. A method losing against shift cannot discard a feasible incumbent. With additional operational predicates, every candidate is filtered and lack of an admissible candidate is reported explicitly.

B. DP State, Actions, and Correctness

For residual segment progress $q \in \prod_v \{0, \dots, h_v\}$ and relative time t , let $\mathcal{A}(t, q)$ contain action sets of size at most m . Every running segment $0 < q_v < h_v$ must be in the action. New segments require release $R_v^d \leq t$, all positive predecessor thresholds $q_u \geq \lambda_{uv}$, and for a zero-lag predecessor either $q_u > 0$ or joint start in the same action. An empty action is allowed only when no segment is running. Then

$$\begin{aligned} q' &= q + \mathbf{1}_A, & V(t, h) &= t, \\ V(t, q) &= \min_{A \in \mathcal{A}(t, q)} V(t + 1, q'). \end{aligned} \quad (22)$$

V returns relative completion time. The implementation explores time layers with one witness per (t, q) and stops at the first complete layer. Running segments are determined by q , so no additional machine identities are needed for this cost-free identical-machine model. A feasible interval set is

subsequently colored onto machines. This compression would be invalid with machine eligibility or switching-history costs.

Theorem 4 (DP exactness and guarded termination). *If the full residual state search completes, DP returns the minimum integer checkpoint makespan. If a state budget stops the search, only a validated incumbent and valid lower bound are returned; optimality is not claimed.*

Proof. Inductively, each transition satisfies releases, lag activation, capacity, and obligatory continuation. It therefore yields a feasible prefix. Conversely, any feasible integer checkpoint schedule induces these actions slot by slot, including idle actions and zero-lag joint starts. Thus the state graph contains every such schedule. Merging identical (t, q) states preserves continuations: started/completed status and running remaining time are determined by q ; labels and prior orders impose no future cost. Breadth-first time layers reach a terminal state at the smallest completion time. For incumbent H , a state at time t cannot improve it if

$$t + \max \left\{ \left\lceil \frac{\sum_v (h_v - q_v)}{m} \right\rceil, \max_v (h_v - q_v) \right\} \geq H. \quad (23)$$

Pruning this state does not remove a strict improvement. If no improving state remains, the incumbent is optimal. A budget interruption does not establish exhaustion, so the implementation reports a state-budget status. $\square$

The nominal score recurrence (3) is a linear DAG DP, while (22) is an exponential scheduling DP. They are not the same computational claim.

C. Local DP for Larger Residuals

The primary experiment uses the full residual problem when $N \leq 7$. Otherwise it selects the last seven nodes of a topological order. All other segments retain the incumbent schedule. Window work starts no earlier than the completion of every frozen outside segment, and its releases include all outside predecessor activation times. A topological suffix has no outgoing arc into the frozen prefix, so this conservative boundary permits concatenation without machine or dependency conflict. DP is exact only in this restricted neighborhood if completed; global optimality is not claimed. A nonimproving local candidate is discarded.

D. LP Relaxation and List Decoding

For a finite feasible horizon H , let x_{vt} represent a segment starting at integer $t \in [R_v^d, H - h_v]$. The time-indexed integer

model is

$$\min z \quad (24)$$

$$\sum_t x_{vt} = 1 \quad \forall v, \quad (25)$$

$$\sum_v \sum_{t: t \leq k < t+h_v} x_{vt} \leq m \quad \forall k, \quad (26)$$

$$\sum_t tx_{vt} - \sum_t tx_{ut} \geq \lambda_{uv} \quad \forall (u, v) \in E_R, \quad (27)$$

$$z \geq h_v + \sum_t tx_{vt} \quad \forall v, \quad (28)$$

$$x_{vt} \in \{0, 1\}. \quad (29)$$

Replacing binaries by $0 \leq x_{vt} \leq 1$ yields a minimization lower bound z_{LP} . The horizon is the best among four feasible list candidates; an optimum is no worse than this horizon, so the restriction does not exclude an optimal integer solution. The solver also receives the analytic lower bound.

Expected starts $\bar{t}_v = \sum_t tx_{vt}$ provide priorities: at each actual dispatch event, the executable segment with smallest $\bar{t}_v$ goes first. This LP-list decoder respects capacity and progress; independent rounding of fractional start times does not. LP-assisted recovery retains the best of shift, residual BOS, and decoded LP-list. Only successful LP solves contribute their objective as a lower bound. Solver failure retains feasible greedy/shift candidates and its status is logged.

LP can also certify that a deadline is impossible when $t_0 + z_{LP} > B$ and can provide valid bounds for branch pruning when the LP reflects that branch's restrictions. This implementation uses LP for a global lower bound and list priorities; it does *not* solve a new LP at each DP state or claim branch-wise LP pruning. The binary model is an independent small-instance oracle, not the production recovery policy.

VI. ALGORITHM AND COMPUTATIONAL COST

Proposition 5 (End-to-end correctness). *With finite releases, the primary-model controller terminates and returns a physically feasible schedule; it labels a schedule accepted or recovered only if its completion is at most B . With additional operational predicates, it returns either an admissible candidate or an explicit no-candidate status.*

Proof. The finite acyclic checkpoint construction terminates. Theorem 2 provides a primary-model feasible shift; Theorem 3 justifies the early return. Otherwise the finite-DAG greedy dispatcher eventually executes every segment after its releases and predecessor thresholds; local DP has a state cap and the LP solver has a time limit with a feasible fallback. Reconstruction and operational filtering validate every selectable candidate. Deadline labels are assigned only after the explicit comparison with B . If filtering removes every candidate, the algorithm reports that outcome rather than returning an illegal schedule. $\square$

There is no LP/DP call on the acceptance branch. Lower bounds in post-hoc experiment certificates do not imply that a solver was executed by an accepted deployment decision.

Algorithm 1 BOS-2L with acceptance-gated recovery**Require:** Nominal BOS plan, observed prefix at t_0 , gates d ,

$$B = D + \tau$$

- 1: Freeze history and form the checkpoint residual graph
- 2: Construct S_{shift} by (7)
- 3: **if** $\text{Valid}_d(S_{\text{shift}})$ and $C_{\text{shift}} \leq B$ **then**
- 4: **return** S_{shift} , ACCEPTEDSHIFT
- 5: **end if**
- 6: Initialize portfolio with admissible shift and residual BOS
- 7: If requested, add a guarded local-DP or LP-list candidate
- 8: Validate candidates against history and operational conditions
- 9: **if** no admissible candidate exists **then**
- 10: **return** NOOPERATIONALCANDIDATE
- 11: **end if**
- 12: Select a candidate of minimum makespan within the portfolio
- 13: **if** its completion is at most B **then**
- 14: **return** candidate, RECOVERED
- 15: **else**
- 16: **return** candidate, DEADLINEMISSED, valid bound if available
- 17: **end if**

BOS scores take $O(|V| + |E|)$; event dispatch with heap operations takes $O((|V| + |E|) \log(|V| + |E|))$ for the inherited core without decision-trace copying. Residual cut generation has size $N = O(|V| + |E| + g)$ for g material gates, plus sorting of cuts. Projection after machine-order construction takes $O(N + |E_R|)$; sorting machine sequences adds $O(N \log N)$. Memory is linear in nodes and arcs. These are core bounds; the independent physical validator scans original edges against reconstructed intervals and can cost $O(N^2 + |E|N)$ in this transparent implementation.

Let $M = \prod_v (h_v + 1)$, horizon H , and $K_m = \sum_{k=0}^{\min(m, N)} \binom{N}{k}$. A conservative bound for the provided DP, including copied path witnesses, is $O(HMK_m(N + |E_R| + H))$ time and $O(HM)$ frontier-witness memory. The state budget caps inserted states but not polynomially the unconstrained problem. Local windows replace M by their window product and do not make global exact scheduling polynomial.

The time-indexed LP has $O(NH)$ variables and $O(N + H + |E_R|)$ rows, with sparse construction potentially $O(NH \max h_v + |E_R|H)$ nonzeros. Its size is pseudopolynomial in numeric time magnitudes. Binary optimization is combinatorial; a time limit is not an optimality certificate. No FPTAS follows from calling a solver or introducing a tolerance parameter.

VII. REPRODUCIBLE EXPERIMENTS

A. Protocol and Metrics

The primary suite contains $n \in \{6, 14\}$, $m \in \{2, 3\}$, overlap targets $\alpha \in \{0, 0.75\}$, and six seeds. Job times are

TABLE I

PRIMARY SUITE: 432 DELIVERY SCENARIOS. ILLEGAL FROZEN PLANS RECEIVE NO COMPLETION CREDIT.

Policy	On time	Calls	Mean C	Tardiness
Frozen BOS	0	0	–	–
Right shift only	156	0	17.736	4.023
Always greedy	200	432	16.250	3.028
Gated greedy	200	276	16.637	3.028
Gated local DP	200	276	16.637	3.028
Gated LP-assisted	200	276	16.634	3.028

uniform integers 1–4. Forward edges appear independently with probability 0.18; odd seeds additionally connect every earlier job to the final sink. Lag is $p_u - \lfloor \alpha p_u \rfloor$. Reveal time is $\max(1, \lfloor C_0/3 \rfloor)$. A deterministic pending job receives a gate at its current progress or one unit later (clipped below completion). Arrival is the later of revelation and nominal gate-reaching time plus $d \in \{2, 6, 12\}$. This gives 144 events. Each is evaluated with $D = C_0$ and $\tau \in \{0, 2, 6\}$, giving 432 paired delivery scenarios.

All six policies share one nominal plan and event. Frozen BOS performs no repair and is reported infeasible when it crosses the late gate; it is not credited with a short but illegal makespan. Right-shift-only never invokes recovery. Always-greedy repairs every event. The three gated portfolios use greedy, DP, or LP assistance only when projection misses B . The DP uses at most 12,000 inserted states and a seven-segment suffix window. LP and oracle limits are 10 and 15 seconds respectively. Full MILP is run on all $n = 6$, seed 0–1 events (24 events), with statuses retained.

We measure physical feasibility, on-time count, recovery calls, absolute completion, tardiness, total segment start deviation from reference, and segment reassignment count. Machine labels affect the latter metric; it measures changes made by this particular dispatcher, not the minimum possible relabeling distance. DP/LP component calculations are cached across the three budgets, then their cost is charged only to triggered decisions. Timing is therefore measured component work, not an end-to-end deployment latency benchmark. Post-hoc oracle and lower-bound costs are excluded from policy timing. All raw component records remain available.

B. Primary Results

Projection accepts 156/432 scenarios (36.11%) without any recovery. Gated greedy raises on-time delivery from 156 to 200 cases, recovering 44 of 276 triggered cases. Gated local DP and LP-assisted recovery reach the same on-time count. Their lack of additional deadline successes is a negative result, not an omitted outcome. LP improves mean completion by only 0.0023 time units over gated greedy in this suite. All feasible policies pass physical validation; all frozen unmodified plans fail because the generator deliberately places an effective late material gate.

Always-greedy has mean completion 16.250 versus 16.637 for gated greedy. This is expected: the latter retains acceptable plans even when a faster alternative exists. Gated greedy

reduces mean segment start deviation from 19.708 to 15.812 and mean reassigned segments from 3.965 to 1.752. Right shift has zero reassignment but larger mean deviation (17.896) because long delays propagate. Thus stability depends on the metric; shift does not minimize every disturbance measure.

The main DP status counts are 71 full bound certificates, one full searched optimum, 46 local bound certificates, and 26 local searched optima. A local status certifies only the conservative suffix subproblem. All 24 sampled exact MILP solves terminate optimally. No LP or oracle failure is hidden in these totals.

C. Harder Small-Instance Suite

A separate fixed sweep uses seeds 928100–928179, seven jobs with processing times 2–7, two or three machines, forward-edge probability 0.10, and lags sampled from zero, half-processing, or full-processing time. Revelation is at time one; the chosen job receives a gate at its current progress, delayed by one to four time units after the later of revelation or nominal gate time. All 80 cases are retained. This suite uses full residual DP with a 30,000-state cap rather than the primary suite’s local window; its results are not pooled with the policy table.

DP improves the best shift/greedy incumbent in 16/80 cases; LP-assisted selection improves it in 7/80. All 80 DP results match independently solved MILP optima: 44 terminate at an initial bound certificate and 36 require search. The maximum inserted-state count is 7,608. No case reaches the configured cap, so these tests do not establish scalability to larger residual graphs. A separate forced-budget unit test verifies that stopping early does not report optimality. Raw unguarded LP-list outcomes, including losses, remain in the stress CSV.

D. Independent Checks and Scope

Every recorded candidate is checked for progress, release, capacity, and prefix consistency. Tests cover a running predecessor halted before its handoff, zero-lag simultaneous activation, exact deadline equality, no optimization on an accepted branch, operational rejection, budget exhaustion, and 24 additional randomized DP–MILP comparisons. Exhausted local search is labeled local; a solver status is not inferred from a numerical coincidence. Shared seeds make delay/slack variants dependent; we do not report unclustered significance tests.

VIII. GUARANTEES THAT DO NOT TRANSFER AUTOMATICALLY

The corrected earlier work-conserving residual list bound remains available for residual greedy and LP-list under the same bounded-lag model. If $R_{\max} = \max_v R_v^d$, the single blocking-chain argument gives

$$C_{\text{rec}}^{\text{list}} \leq \frac{W_R}{m} + \left(1 - \frac{1}{m}\right) L_{R,\lambda} + \frac{R_{\max}}{m}. \quad (30)$$

The proof uses $C \leq W_R/m + \rho + (1 - 1/m)Q$, where a single blocking chain starts at release ρ , $\rho + Q \leq L_{R,\lambda}$ and

$\rho \leq R_{\max}$. Consequently, for residual lower bound $L > 0$, a sufficient condition for $(2 - 1/m + \varepsilon)$ relative to the same-prefix residual optimum is $R_{\max} \leq m\varepsilon L$. Here ε is dimensionless. This condition is not a promised small rounding loss.

An accepted right shift can deliberately idle despite executable work, so its guarantee is (15) and $C_{\text{shift}} \leq B$, not that work-conserving ratio. A triggered portfolio that keeps a valid list candidate is no worse in makespan than that candidate in the primary model. Added operational filters may remove the candidate, invalidating this inference. None of these statements compares against an unrestricted clairvoyant schedule that can alter executed history or change checkpoint permissions.

IX. LIMITATIONS AND NEXT RESEARCH QUESTIONS

The model permits free checkpoint suspension and migration, one revealed material event, deterministic arrivals thereafter, and identical machines. It does not prove robustness under repeated unknown events, stochastic service guarantees, restart losses, machine calendars, or arbitrary operational constraints. Such predicates can trigger recovery through the interface, but the current candidate generators need not find a feasible schedule under them.

The main empirical advantage is fewer recovery decisions and lower measured plan change, not improved makespan against always-greedy. A production objective might impose a maximum number of reassignments, a weighted stability budget, or a lexicographic objective that first meets B and then minimizes disturbance. The current recovery portfolio minimizes makespan among its few candidates only after the tolerance test fails. It does not solve that richer stability optimization.

Future evaluation needs public benchmarks or production traces, realistic switching costs, matched computation budgets, and critical-chain local windows. Novelty and industrial generalization remain unestablished.

X. CONCLUSION

BOS-2L preserves acceptable delayed plans and invokes recovery selectively. Its projection envelope, acceptance test, and comparable checkpoint solvers provide auditable guarantees under explicit operational assumptions.

REFERENCES

- [1] G. E. Vieira, J. W. Herrmann, and E. Lin, “Rescheduling manufacturing systems: A framework of strategies, policies, and methods,” *Journal of Scheduling*, vol. 6, pp. 39–62, 2003.
- [2] V. Subramaniam, A. S. Raheja, and K. R. B. Reddy, “Reactive repair tool for job shop schedules,” *International Journal of Production Research*, vol. 43, no. 1, pp. 1–23, 2005.
- [3] M. Bartusch, R. H. Möhring, and F. J. Radermacher, “Scheduling project networks with resource constraints and time windows,” *Annals of Operations Research*, vol. 16, pp. 199–240, 1988.
- [4] R. L. Graham, “Bounds on multiprocessing timing anomalies,” *SIAM Journal on Applied Mathematics*, vol. 17, no. 2, pp. 416–429, Mar. 1969.
- [5] R. Kolisch, “Serial and parallel resource-constrained project scheduling methods revisited: Theory and computation,” *European Journal of Operational Research*, vol. 90, no. 2, pp. 320–333, 1996.